# DCS 2026 Spring
# Lab09 – Code Review

TA : 陳孟頡　Advisor : Tian-Sheuan Chang

Date : 2026.05.19

# Outline

- Lab Review
- FAQ
- Reference Code

# Lab09 : Asynchronous FIFO

- Fill the blanks in AsyncFIFO.sv to complete the Asynchronous FIFO.

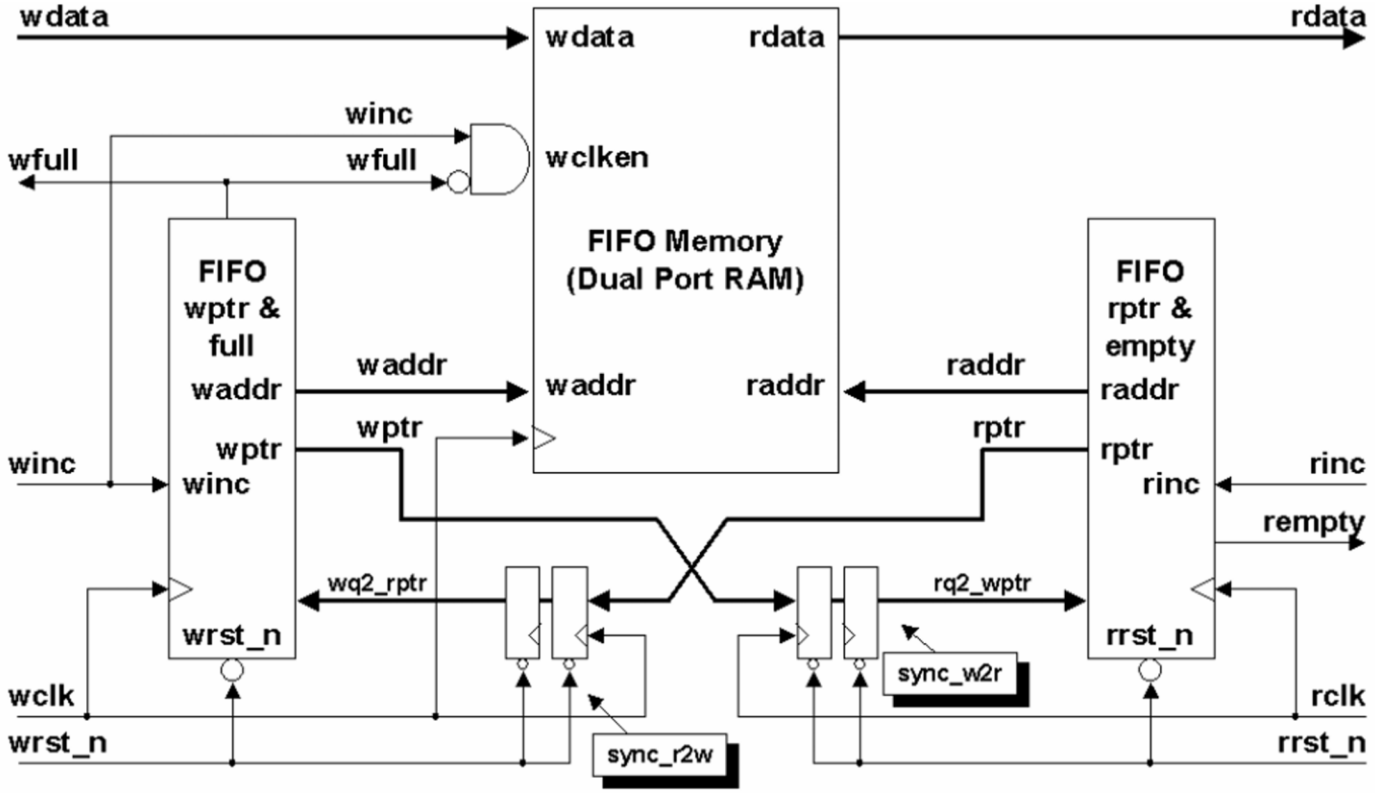
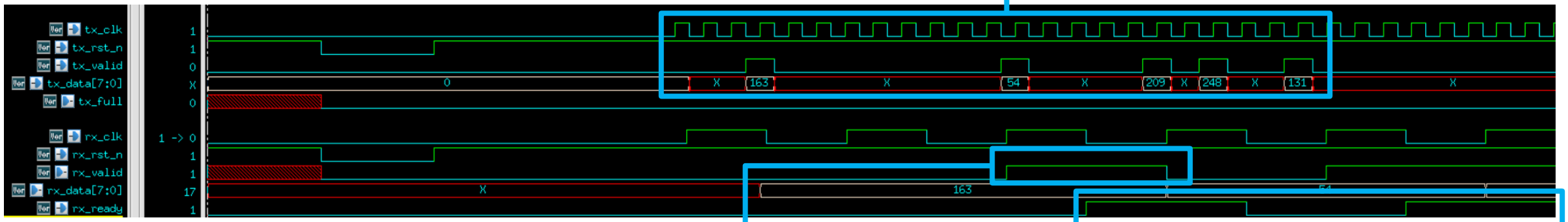



Figure 5 - FIFO1 partitioning with synchronized pointer comparison

# Lab09 : Asynchronous FIFO

- Specifications:
  - tx_clk = 2.5ns (write domain), rx_clk = 14.1ns (read domain)

TX domain randomly
send write data



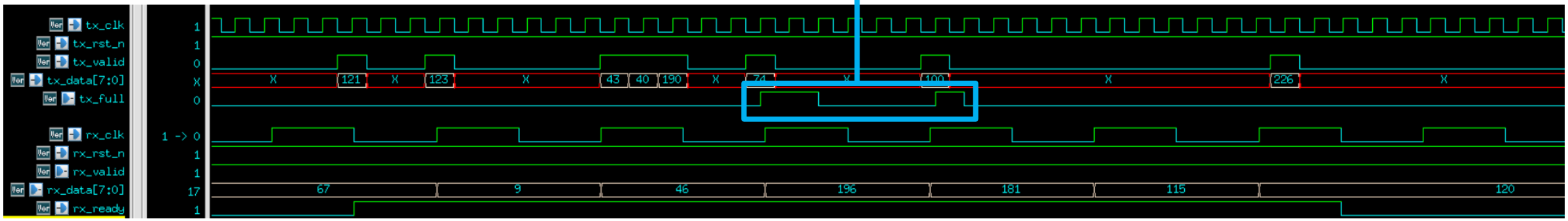

rx_valid should be high
when rx_data is valid

RX domain randomly set
ready signal

# Lab09 : Asynchronous FIFO

- Specifications:
  - tx_clk = 2.5ns (write domain), rx_clk = 14.1ns (read domain)

tx_full should pull high
when FIFO full



# Outline

- Lab Review
- FAQ
- Reference Code

# Address Bit Number

| Address | Data |
|---|---|
| 7 | **Data 7** |
| 6 | **Data 6** |
| 5 | **Data 5** |
| 4 | **Data 4** |
| 3 | **Data 3** |
| 2 | **Data 2** |
| 1 | **Data 1** |
| 0 | **Data 0** |

waddr[3:0] = 4'b1000

raddr[3:0] = 4'b0000

Dual-port RAM

EMPTY
if (waddr == raddr)

FULL
if (waddr == {~raddr[3], raddr[2:0]})

# Address Bit Number

- Data Write/Read:
  - Use binary pointer.
  - Data address bit number = PTR_WIDTH -1 = ADDR_WIDTH.

```
// Data write
always @(posedge tx_clk) begin
    if (w_en) begin
        mem[w_ptr_bin[ADDR_WIDTH-1:0]] <= tx_data;
    end
end
```

```
// Data Read
assign rx_data  = mem[r_ptr_bin[ADDR_WIDTH-1:0]];
assign rx_valid = !rx_empty;
```

# Gray-Coded Pointer Full Condition

- **Single-Bit Flip**: Only one bit changes between consecutive values.
- **Simple Logic**: Easily converted from binary using a single XOR shift operation: $G = B \oplus (B \gg 1)$

How to determine **full** condition if using gray-coded pointer instead of binary?

| Decimal | Binary | Gray Code |
|---|---|---|
| 0 | 000 | 000 |
| 1 | 001 | 001 |
| 2 | 010 | 011 |
| 3 | 011 | 010 |
| 4 | 100 | 110 |
| 5 | 101 | 111 |
| 6 | 110 | 101 |
| 7 | 111 | 100 |

# Gray-Coded Pointer Full Condition

- Full Condition:
  - Compare write gray-coded pointer with read gray-coded sync pointer.
  - Flip the highest 2bits.

```
// ============================================================
// Full / Empty
// ============================================================
assign tx_full = (w_ptr_gray ==
                  {~r_ptr_gray_sync2[PTR_WIDTH-1:PTR_WIDTH-2], r_ptr_gray_sync2[PTR_WIDTH-3:0]});

assign rx_empty = (r_ptr_gray == w_ptr_gray_sync2);
```

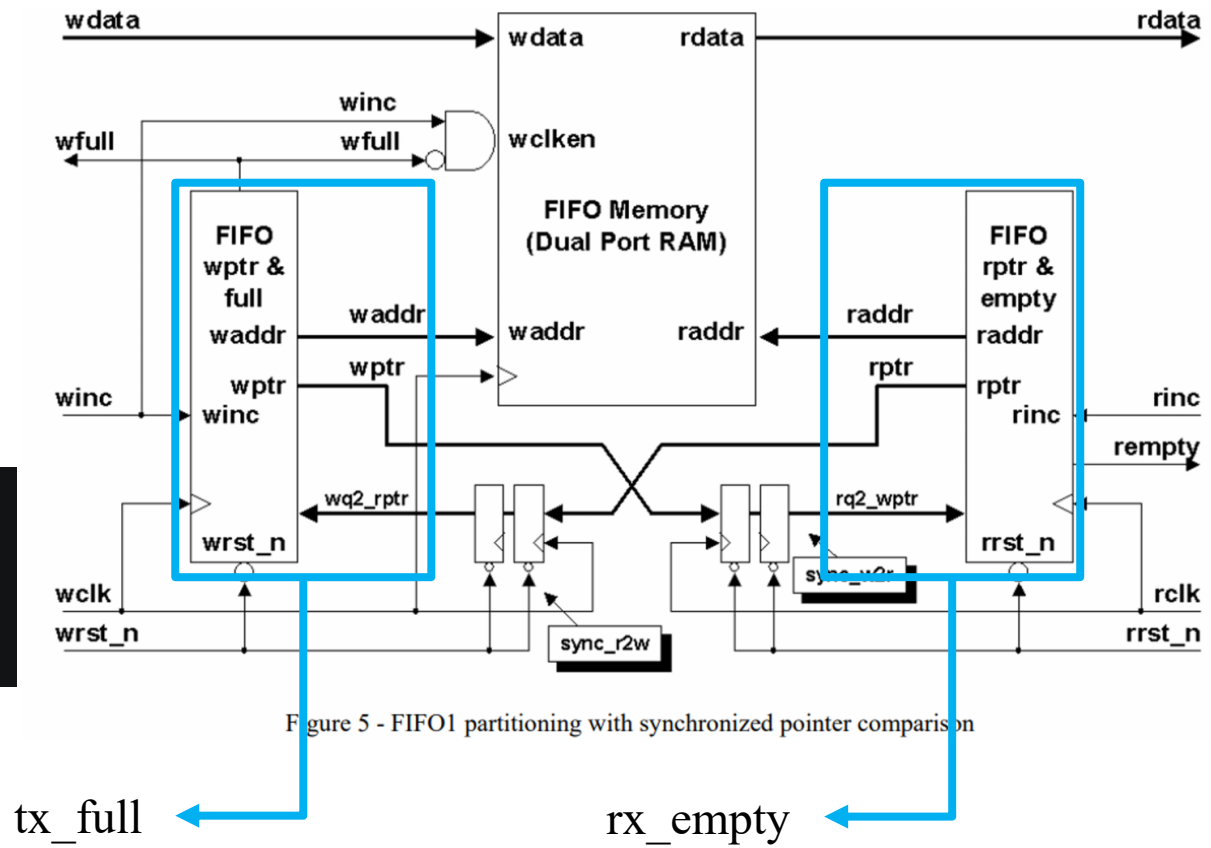



Figure 5 - FIFO1 partitioning with synchronized pointer comparison

# Outline

- Lab Review
- FAQ
- Reference Code

# Reference Code

```
module AsyncFIFO #(
    parameter DATA_WIDTH = 8,
    parameter ADDR_WIDTH = 4
)(
    // TX Domain
    input  logic                      tx_clk,
    input  logic                      tx_rst_n,
    input  logic                      tx_valid,
    input  logic    [DATA_WIDTH-1:0]  tx_data,
    output logic                      tx_full,

    // RX Domain
    input  logic                      rx_clk,
    input  logic                      rx_rst_n,
    output logic                      rx_valid,
    output logic    [DATA_WIDTH-1:0]  rx_data,
    input  logic                      rx_ready
);
    localparam PTR_WIDTH = ADDR_WIDTH + 1;
    localparam MEM_DEPTH = 1 << ADDR_WIDTH;

    // ------------------------------------------------
    // Internal Signals
    // ------------------------------------------------
    logic   [DATA_WIDTH-1:0]    mem         [0:MEM_DEPTH-1];

    // Write Pointers
    logic   [PTR_WIDTH-1:0]     w_ptr_bin, w_ptr_gray;
    logic   [PTR_WIDTH-1:0]     w_ptr_bin_next, w_ptr_gray_next;

    // Read Pointers
    logic   [PTR_WIDTH-1:0]     r_ptr_bin, r_ptr_gray;
    logic   [PTR_WIDTH-1:0]     r_ptr_bin_next, r_ptr_gray_next;
```

```
// Synchronizers (Double Flop)
logic   [PTR_WIDTH-1:0]     w_ptr_gray_sync1, w_ptr_gray_sync2; // Write ptr synced to RX domain
logic   [PTR_WIDTH-1:0]     r_ptr_gray_sync1, r_ptr_gray_sync2; // Read ptr synced to TX domain

logic                       w_en;
logic                       r_en;
logic                       rx_empty;


// ============================================================
// Write Operation
// ============================================================
// write enable when User write && FIFO not full
assign w_en = tx_valid && !tx_full;

// Binary Pointer Update
assign w_ptr_bin_next = w_ptr_bin + (w_en ? 1'b1 : 1'b0);

// Gray Pointer Update
assign w_ptr_gray_next = (w_ptr_bin_next >> 1) ^ w_ptr_bin_next;

always @(posedge tx_clk or negedge tx_rst_n) begin
    if (!tx_rst_n) begin
        w_ptr_bin   <= {PTR_WIDTH{1'b0}};
        w_ptr_gray  <= {PTR_WIDTH{1'b0}};
    end
    else begin
        w_ptr_bin   <= w_ptr_bin_next;
        w_ptr_gray  <= w_ptr_gray_next;
    end
end

// Data write
always @(posedge tx_clk) begin
    if (w_en) begin
        mem[w_ptr_bin[ADDR_WIDTH-1:0]] <= tx_data;
    end
end
```

# Reference Code

```
// ============================================================
// Read Operation
// ============================================================
// Read enable when User is ready && FIFO not empty
assign r_en = rx_ready && !rx_empty;

// Binary Pointer Update
assign r_ptr_bin_next = r_ptr_bin + (r_en ? 1'b1 : 1'b0);

// Gray Pointer Update
assign r_ptr_gray_next = (r_ptr_bin_next >> 1) ^ r_ptr_bin_next;

always @(posedge rx_clk or negedge rx_rst_n) begin
    if (!rx_rst_n) begin
        r_ptr_bin   <= {PTR_WIDTH{1'b0}};
        r_ptr_gray  <= {PTR_WIDTH{1'b0}};
    end else begin
        r_ptr_bin   <= r_ptr_bin_next;
        r_ptr_gray  <= r_ptr_gray_next;
    end
end

// Data Read
assign rx_data  = mem[r_ptr_bin[ADDR_WIDTH-1:0]];
assign rx_valid = !rx_empty;
```

```
    // ============================================================
    // Sync
    // ============================================================
    // Sync Read Pointer to TX Domain
    always @(posedge tx_clk or negedge tx_rst_n) begin
        if (!tx_rst_n) begin
            r_ptr_gray_sync1 <= {PTR_WIDTH{1'b0}};
            r_ptr_gray_sync2 <= {PTR_WIDTH{1'b0}};
        end else begin
            r_ptr_gray_sync1 <= r_ptr_gray;
            r_ptr_gray_sync2 <= r_ptr_gray_sync1;
        end
    end

    // Sync Write Pointer to RX Domain
    always @(posedge rx_clk or negedge rx_rst_n) begin
        if (!rx_rst_n) begin
            w_ptr_gray_sync1 <= {PTR_WIDTH{1'b0}};
            w_ptr_gray_sync2 <= {PTR_WIDTH{1'b0}};
        end else begin
            w_ptr_gray_sync1 <= w_ptr_gray;
            w_ptr_gray_sync2 <= w_ptr_gray_sync1;
        end
    end

    // ============================================================
    // Full / Empty
    // ============================================================
    assign tx_full = (w_ptr_gray == {~r_ptr_gray_sync2[PTR_WIDTH-1:PTR_WIDTH-2], r_ptr_gray_sync2[PTR_WIDTH-3:0]});

    assign rx_empty = (r_ptr_gray == w_ptr_gray_sync2);

endmodule
```

Thank you.
Any questions?